

ARM Processor: Initial vs Final Verilog Implementation

Line-by-Line Code Comparison | Arty A7-100T FPGA Implementation

Initial Code

Sourced from GitHub – Multi-cycle ARM processor in Verilog. Contained logic errors, wrong ALU op-codes, missing resets, `\$readmemh` incompatibility with Vivado, and type mismatches causing simulation failures.

Final Code

Fully corrected implementation. Simulated successfully on EDA Playground & Vivado. Synthesized and implemented on Arty A7-100T FPGA with button-step clock, debounce logic, and LED debug output.

Modules (10)

top · cpu · controller · decoder · condlogic · datapath · alu
· regfile · imem · dmem
+ adder · mux2 · flopr · flopenr · extend (support modules)

Summary of All Errors in Initial Code

ALU	ALU operation codes were SWAPPED — ADD/SUB mapped to AND/OR slots and vice versa
ALU	Carry & Overflow flags computed with wrong formula; assign inside always block caused latch
Decoder	Branch instruction: ALUSrc=0 (used register instead of ExtImm as SrcB) → wrong branch target
Decoder	RegSrc bits for branch not set → PC not fed correctly as SrcA
Decoder	default ALUControl = 2'bx caused undefined simulation behaviour
condlogic	Outputs declared as 'output reg' → synthesiser inferred unwanted latches
condlogic	CondEx declared as 'reg' → used in assign statement causing mixed-signal error
datapath	Output ports (ALUFlags, PC, ALUResult, WriteData) declared as 'output reg' → driver conflict
datapath	mux2 / flopr instantiated without generics WIDTH bound; port order wrong
regfile	No reset port → registers held X on power-up; wa3=R15 write not blocked
imem	\$readmemh() fails in Vivado during synthesis (file not found in FPGA flow)
dmem	output rd declared as 'reg' → illegal; RAM uninitialized → X propagation
flopr	Non-parameterised (fixed 32-bit) → reuse impossible; sensitivity list used comma instead of 'or'
top	Single clk/reset ports → unusable for FPGA; no debounce, no LED I/O, no constraint file
testbench	clk not initialized (X at t=0) → clock never toggles; expected value message wrong (said 7 not 11)

Module 01

alu.v

Arithmetic Logic Unit – Operation Code & Flag Computation Fixes

Module 01 – alu.v | Operation Code Mapping (Critical Bug)

✗ INITIAL (GitHub – Buggy)

```
2'b00: begin //AND
    {Carry, ALUResult} = SrcA + SrcB; // ← does ADD not AND!
```

```
2'b01: begin //OR
    {Carry, ALUResult} = SrcA - SrcB; // ← does SUB not OR!
```

```
2'b10: begin //ADD
    ALUResult = SrcA & SrcB; // ← actually AND!
```

```
2'b11: begin //SUB
    ALUResult = SrcA | SrcB; // ← actually OR!
```

```
assign Zero = (ALUResult==32'b0)?1:0;
assign Negative = ALUResult[31];
// Zero/Neg outside always → latch risk
```

```
Overflow=((SrcA[31]==SrcB[31])&&
    (ALUResult[31]!=SrcA[31]));// wrong for SUB
```

✓ FINAL (Fixed – Working)

```
2'b00: begin // ADD
    sum = {1'b0,SrcA}+{1'b0,SrcB}; // 33-bit ADD
    ALUResult = sum[31:0]; Carry = sum[32];
```

```
2'b01: begin // SUB
    sum = {1'b0,SrcA}-{1'b0,SrcB};
    Carry = ~sum[32]; // ARM: Carry=1 when no borrow
```

```
2'b10: begin // AND
    ALUResult = SrcA & SrcB;
```

```
2'b11: begin // OR
    ALUResult = SrcA | SrcB;
```

```
// All flags inside always block:
Zero      = (ALUResult == 32'b0);
Negative = ALUResult[31];
ALUFlag   = {Negative,Zero,Carry,Overflow};
```

```
// ADD Overflow:
Overflow=(~SrcA[31]&~SrcB[31]& ALUResult[31])
    | ( SrcA[31]& SrcB[31]&~ALUResult[31]);
```

Module 01 – alu.v | Why These Changes Fixed the ALU

Bug 1 – Swapped Op-codes

The initial code had ADD logic under case 2'b00 (labelled AND) and AND logic under 2'b10 (labelled ADD). Every arithmetic instruction executed the wrong operation, making the CPU completely non-functional.

Fix – Correct Mapping

Final code maps 2'b00→ADD, 2'b01→SUB, 2'b10→AND, 2'b11→OR – matching the decoder's ALUControl encoding. This single fix makes all data-processing instructions produce correct results.

Bug 2 – Carry/Overflow Flag Logic

Initial code used a 32-bit concatenation {Carry,ALUResult} which drops the carry bit into ALUResult[31], corrupting the result. Overflow formula was also wrong for subtraction.

Fix – 33-bit Arithmetic

A dedicated 33-bit reg 'sum' captures the carry-out cleanly (sum[32]). For SUB, ARM convention requires Carry = NOT borrow, so Carry = ~sum[32]. Overflow uses separate correct formulas for ADD and SUB.

Bug 3 – assign outside always

Zero and Negative were driven by continuous assign statements while also being declared inside the always block, creating multiple-driver conflicts and simulation warnings.

Fix – Single always block

All flag computations (Zero, Negative, Carry, Overflow, ALUFlag) are computed inside one always@(*) block, eliminating driver conflicts and making the code synthesisable cleanly.

Module 02

decoder.v

Instruction Decoder – Branch Fix & Default Value Safety

Module 02 – decoder.v | Branch Instruction Decoding (Critical)

✗ INITIAL (GitHub – Buggy)

```
2'b10: controls = 10'b1001100010; // B
// Packed: Branch=1,MemtoReg=0,MemW=0
//         ALUSrc=1,ImmSrc=10,RegW=0
//         RegSrc=00,ALUOp=0
// RegSrc=00 → RA1=Instr[19:16] NOT PC!
```

```
// Default controls = 10'bx
// X propagates through entire datapath!
default: controls = 10'bx;
```

```
// ALU decoder default:
default: ALUControl = 2'bx;
// X causes undefined simulation output
```

```
// PCS computed outside always block
assign PCS=( (Rd==4'b1111)&RegW) |Branch;
```

```
// casex used → accidental X/Z matching
casex(Op)
```

✓ FINAL (Fixed – Working)

```
2'b10: begin // Branch
    Branch = 1;
    ALUSrc = 1; // FIX: use ExtImm as SrcB
    ImmSrc = 2'b10;
    RegSrc = 2'b01; // FIX: RA1=R15(PC+8) as SrcA
end
```

```
// All signals initialized to 0 at start:
Branch=0; MemtoReg=0; MemW=0; ALUSrc=0;
ImmSrc=2'b00; RegW=0; RegSrc=2'b00; ALUOp=0;
```

```
// Safe default:
default: ALUControl = 2'b00; // ADD = safe no-op
```

```
// PCS computed INSIDE the always block:
PCS = ((Rd==4'b1111) & RegW) | Branch;
// Avoids potential race with Branch reg
```

```
// case used → clean, deterministic
case(Op)
```


Module 02 – decoder.v | Why These Changes Fixed Branching

Bug 1 – RegSrc=00 for Branch (Wrong SrcA)

ARM branch adds the signed offset to PC+8. SrcA must be R15 (PC+8). With RegSrc=2'b00, RA1=Instr[19:16] (an arbitrary register), so the branch target was calculated from a wrong base address.

Fix – RegSrc=2'b01

Setting RegSrc[0]=1 routes RA1 to R15 (the constant 4'b1111 path in the RA1 mux), which feeds PC+8 as SrcA. The ALU then computes PC+8+offset = correct branch target.

Bug 2 – X Propagation from defaults

default: ALUControl=2'bx causes unknown ('X') values to propagate through the entire datapath in simulation, making all signal waveforms show X after the first undefined instruction.

Fix – Zero initialization of all signals

Every control signal is set to 0 at the top of the always block before the case statement. This means any unrecognised opcode safely produces a NOP-like behaviour instead of corrupting state.

Bug 3 – casex hazard

casex treats X and Z bits as don't-cares in the select expression. If Op is momentarily X during simulation startup, casex can match an unintended case, causing spurious control signal assertions.

Fix – plain case

Using plain case ensures only exact matches trigger. Combined with default-zero initialization, the decoder is deterministic from cycle 0 of simulation and synthesises without latch warnings.

Module 03

condlogic.v

Conditional Logic – Output Type & CondEx Driver Fix

Module 03 – condlogic.v | Port Type & CondEx Declaration Fixes



INITIAL (GitHub – Buggy)

```
output reg PCSrc, RegWrite, MemWrite
// 'reg' on module output → infers latches
// Vivado DRC error: multi-driven net
```

```
reg CondEx; // reg driven by submodule output
// condcheck drives CondEx via port
// reg + port assignment = multiple drivers!
```

```
// No timescale directive
// EDA Playground simulation unit mismatch
```

```
// input logic clk, reset
// 'logic' keyword - SystemVerilog only!
// Fails in Verilog-2001 mode
```

```
flopenr #(.WIDTH(2)) flagreg1(...)
// Named parameter - works but verbose
```



FINAL (Fixed – Working)

```
output PCSrc, RegWrite, MemWrite
// Plain wire outputs driven by assign
// Clean synthesis, no latches inferred
```

```
wire CondEx; // wire → single driver from condcheck
// condcheck cc(Cond, Flags, CondEx);
// No conflict, simulator/synth happy
```

```
`timescale 1ns / 1ps
// Explicit simulation time unit
// Matches all other modules
```

```
input clk,
input reset,
// Standard Verilog 2001 port types
// Compatible with all tool modes
```

```
flopenr #(2) flagreg1(...)
// Positional parameter - cleaner, matches
// parameterised module definition
```

Module 03 – condlogic.v | Why These Changes Fixed Conditional Execution

Bug 1 – output reg on combinational outputs

RegWrite, MemWrite, PCSrc are driven by assign statements. Declaring them 'output reg' means Verilog expects them to be driven inside an always block, but assign also drives them. Two drivers → synthesis error and simulation X.

Fix – output (wire)

Removing 'reg' makes them simple wire outputs. The three assign statements (RegW&CondEx, MemW&CondEx, PCS&CondEx) are the sole drivers. Synthesis is clean with no latches and no multiple-driver DRC errors.

Bug 2 – reg CondEx driven by submodule port

condcheck module drives its output 'CondEx' port. If condlogic declares CondEx as reg, two drivers exist (the reg and the port connection). This is illegal in Verilog and causes 'cannot be driven by both procedural and continuous assignment' error.

Fix – wire CondEx

Declaring CondEx as wire allows the condcheck instance to drive it through its output port exclusively. All downstream assigns use it as a read-only value, which is the correct Verilog model.

Bug 3 – input logic (SystemVerilog keyword)

The keyword 'logic' is only valid in SystemVerilog (.sv files). Using it in a .v file in Verilog-2001 mode causes a syntax error in Vivado and EDA Playground when the tool is set to Verilog mode.

Fix – plain input/output

All ports use standard Verilog 2001 'input'/'output' keywords. The module compiles cleanly in both Verilog and SystemVerilog modes, ensuring cross-tool compatibility.

Module 04

datapath.v

Datapath – Output Types, Mux/Flop Generics & Regfile Reset

Module 04 – datapath.v | Output Port & Instantiation Fixes

✗ INITIAL (GitHub – Buggy)

```
output reg[3:0] ALUFlags,  
output reg[31:0] PC,  
output reg[31:0] ALUResult, WriteData,  
// reg outputs → multiple driver error
```

```
mux2 #(.WIDTH(32)) pcmux(.y(PCNext),  
    .d0(PCPlus4),.d1(Result),.s(PCSrc));  
// Named ports - but d0/d1 order matters
```

```
flopr pcreg(clk,reset,PCNext,PC);  
// Non-parameterised flopr (fixed 32-bit)  
// Reuse impossible for other widths
```

```
adder pcadd1(PC, 32'b100, PCPlus4);  
// 32'b100 = 4 in binary – correct but  
// misleading; tool warning possible
```

```
regfile rf(clk,RegWrite,RA1,RA2,  
    Instr[15:12],Result,PCPlus8,  
    SrcA,WriteData);  
// No reset port → X on powerup
```

✓ FINAL (Fixed – Working)

```
output [3:0] ALUFlags,  
output [31:0] PC,  
output [31:0] ALUResult,  
output [31:0] WriteData, // plain wire
```

```
mux2 #(32) pcmux(PCPlus4, Result, PCSrc, PCNext);  
// Positional: d0=PCPlus4, d1=Result  
// s=PCSrc, y=PCNext – matches module def
```

```
flopr #(32) pcreg(clk, reset, PCNext, PC);  
// WIDTH=32 passed explicitly  
// Same parameterised flopr used everywhere
```

```
adder pcadd1(PC, 32'd4, PCPlus4);  
// 32'd4 – decimal literal, clearer intent
```

```
regfile rf(.clk(clk),.we3(RegWrite),  
    .reset(reset), // ← reset connected!  
    .ra1(RA1),.ra2(RA2),.wa3(Instr[15:12]),  
    .wd3(Result),.r15(PCPlus8),.rd1(SrcA),.rd2(WriteData));
```

Module 04 – datapath.v | Why These Changes Fixed the Datapath

Bug 1 – output reg on wire-driven ports

PC, ALUResult, WriteData, ALUFlags are driven by submodule output ports (flopr, alu etc.). Declaring them 'output reg' creates a second driver conflict. Vivado reports 'multi-driven net' and simulation produces X on those signals.

Fix – plain output (wire)

All datapath outputs are now plain wires. They are driven exclusively by the connected submodule instances. This eliminates all multiple-driver errors and is the correct Verilog structural coding style.

Bug 2 – Non-parameterised flopr

The initial flopr module was fixed at 32 bits with no parameter. When datapath instantiates it without #(32), the default WIDTH (8) from the new parameterised version would be used, causing a width mismatch on the PC register.

Fix – flopr #(32)

Passing #(32) explicitly sets WIDTH=32, matching the 32-bit PCNext/PC wires. The same parameterised flopr is reused in flopenr and other places, keeping the codebase DRY and consistent.

Bug 3 – Regfile without reset, no wa3 guard

Without reset, all 15 registers contain X at startup. X propagates through the ALU and PC logic, preventing any meaningful simulation waveform from appearing. Also, writing to wa3=R15 (4'b1111) was not blocked.

Fix – reset port + wa3 guard

The new regfile has a reset input that zeroes all registers on assertion. The write condition adds 'wa3 != 4'b1111' to prevent accidental corruption of the R15 read-through path (PC+8 is always returned directly).

Module 05

controller.v

Controller – Output Type & Timescale Fixes

Module 05 – controller.v | Declaration & Include Fixes



INITIAL (GitHub – Buggy)

```
output reg [1:0] RegSrc,
output reg RegWrite,
output reg [1:0] ImmSrc,
output reg ALUSrc,
output reg [1:0] ALUControl,
output reg MemWrite, MemtoReg,
output reg PCSrc

`include "decoder.v"
`include "condlogic.v"
// Include chains cause duplicate module
// definition errors in Vivado project flow
```

```
module controller(input logic clk, reset,
// 'logic' → SystemVerilog keyword
// Fails in Verilog-2001 mode
```

```
// No timescale
// Simulation time unit undefined
// Causes 'timescale not set' warnings
```



FINAL (Fixed – Working)

```
output [1:0] RegSrc,
output RegWrite,
output [1:0] ImmSrc,
output ALUSrc,
output [1:0] ALUControl,
output MemWrite, MemtoReg,
output PCSrc

// No includes – Vivado project adds all
// .v files to the project source list.
// Each module compiled once, no duplicates.
`timescale 1ns / 1ps // Added
```

```
module controller(
    input clk, reset,
    // Standard Verilog 2001
    // Works in all tool modes
```

```
`timescale 1ns / 1ps
// Standard timescale header
// All modules now consistent
```

Module 05 – controller.v | Why These Changes Fixed the Controller

Bug 1 – output reg on structurally-driven ports

Controller outputs (RegSrc, RegWrite, ImmSrc, etc.) are wired directly from decoder and condlogic submodule outputs. Declaring them 'output reg' creates the same multi-driver conflict seen in datapath — two things drive the same net.

Fix – plain output wires

Removing 'reg' makes controller outputs transparent pass-throughs from its submodules. Decoder drives ImmSrc, RegSrc, ALUControl etc., condlogic drives PCSrc, RegWrite, MemWrite. One driver per net — synthesis clean.

Bug 2 – `include chains

Using `include "decoder.v" inside controller.v, and `include "controller.v" inside cpu.v creates a chain. When Vivado compiles all project files, each included file gets compiled again, causing 'module already defined' errors.

Fix – Remove all includes

In Vivado's project-based flow, all .v files are added to the project source list. The elaboration tool resolves module instances by name across all source files automatically. No includes needed or desired.

Bug 3 – Missing timescale

Modules without `timescale directives inherit the timescale of the last compiled file, which is non-deterministic. This caused inconsistent delay simulation and 'timescale not set' warnings in EDA Playground.

Fix – Consistent `timescale 1ns/1ps

Every module in the final design has `timescale 1ns/1ps at the top. This gives all simulation delays a consistent reference unit and precision, eliminating timing warnings in both EDA Playground and Vivado simulator.

Module 06

regfile.v

Register File – Reset Port & Write Guard Addition

Module 06 – regfile.v | Reset & Write Protection

✗ INITIAL (GitHub – Buggy)

```
module regfile(  
    input clk, we3,  
    input [3:0] ra1,ra2,wa3,  
    input [31:0] wd3,r15,  
    output [31:0] rd1,rd2);  
// No reset → X at startup  
always @(posedge clk)  
    if (we3) rf[wa3] <= wd3;  
// No reset branch → X survives  
// No R15 write guard → corruption possible
```

```
reg [31:0] rf[14:0];  
// No integer variable for loop  
// Cannot do reset loop in Verilog
```

```
assign rd1=(ra1==4'b1111)?r15:rf[ra1];  
assign rd2=(ra2==4'b1111)?r15:rf[ra2];  
// Combinational reads: correct, kept
```

✓ FINAL (Fixed – Working)

```
module regfile(  
    input clk, we3,  
    input reset, // ← NEW  
    input [3:0] ra1,ra2,wa3,  
    input [31:0] wd3,r15,  
    output [31:0] rd1,rd2);  
always @(posedge clk or posedge reset) begin  
    if (reset)  
        for(i=0;i<15;i=i+1) rf[i]<=32'b0;  
    else if (we3 && wa3!=4'b1111)  
        rf[wa3] <= wd3;  
end
```

```
reg [31:0] rf[14:0];  
integer i; // ← needed for reset for-loop
```

```
assign rd1=(ra1==4'b1111)?r15:rf[ra1];  
assign rd2=(ra2==4'b1111)?r15:rf[ra2];  
// Unchanged - correct read-through logic
```

Module 07

imem.v

Instruction Memory – \$readmemh Replaced with Hardcoded Init

Module 07 – imem.v | Memory Initialization Method



INITIAL (GitHub – Buggy)

```
module imem(  
    input logic [31:0] a,  
    output logic [31:0] rd);  
// 'logic' keyword → SV only!  
  
reg [31:0] RAM[63:0];  
initial  
    $readmemh("memfile.dat", RAM);  
// Vivado synthesis: file not found!  
// Cannot embed .dat in bitstream
```

```
assign rd = RAM[a[31:2]];  
// Word-aligned: correct  
// But RAM is uninitialized = X
```

```
// No timescale directive
```



FINAL (Fixed – Working)

```
module imem(  
    input  [31:0] a,  
    output [31:0] rd);  
// Standard Verilog 2001 types  
reg [31:0] mem[0:63];  
integer i;  
initial begin  
    for(i=0;i<64;i=i+1) mem[i]=32'hE1A00000; // NOP  
    mem[0]=32'hE04F000F; mem[1]=32'hE2802005;  
    // ... all 23 instructions hardcoded  
`
```

```
assign rd = mem[a[31:2]];  
// Same word-aligned access  
// mem is fully initialized → no X
```

```
`timescale 1ns / 1ps  
// Added for consistency
```

Module 07 – imem.v | Why \$readmemh Fails on FPGA & the Fix

Bug – \$readmemh in synthesis

\$readmemh is a simulation-only system task. During Vivado synthesis, the tool cannot locate 'memfile.dat' (it's not part of the netlist), causing 'file not found' critical warnings and leaving RAM as all-X in the synthesised design.

Fix – Inline initial block with hardcoded values

All 23 ARM machine-code instructions are assigned directly in an initial begin block. Vivado synthesises this into a Block RAM or distributed ROM with the values pre-loaded — guaranteed correct on every power-up.

Bug 2 – Uninitialized memory → X propagation

Even in simulation, if \$readmemh fails silently (wrong path), all RAM words are X. The first instruction fetch returns X, which propagates through the entire datapath, making all signal waveforms show X.

Fix – NOP fill + explicit instruction load

The initial block first fills all 64 entries with MOV R0,R0 (NOP = 0xE1A00000), then overwrites entries 0–22 with the program instructions. Any PC overrun fetches a safe NOP instead of X.

Bug 3 – logic keyword

'input logic' and 'output logic' are SystemVerilog constructs. Using them in a .v file causes syntax errors in strict Verilog-2001 mode, which some EDA Playground configurations use by default.

Fix – Standard Verilog 2001 port types

Ports changed to plain 'input [31:0] a' and 'output [31:0] rd' (implicit wire). Compatible with all Verilog tool configurations.

Module 08

dmem.v

Data Memory – Output Type & Initialization Fix

Module 08 – dmem.v | rd Declaration & RAM Init



INITIAL (GitHub – Buggy)

```
output reg [31:0] rd);
// 'reg' on output driven by assign
// Illegal in Verilog: assign + reg conflict
```

```
reg [31:0] RAM[63:0];
assign rd = RAM[a[31:2]];
always@(posedge clk)
    if(we) RAM[a[31:2]] <= wd;
// RAM contents: all X at startup
```

```
// No timescale
```



FINAL (Fixed – Working)

```
output [31:0] rd);
// Plain wire driven by continuous assign
// Correct and synthesisable
reg [31:0] RAM[63:0];
integer i;
initial begin
    for(i=0;i<64;i=i+1) RAM[i]=i; // safe init
end
assign rd = RAM[a[31:2]];
always@(posedge clk)
```

```
`timescale 1ns / 1ps
```

Module 09

flopr.v

Flip-Flop with Reset – Parameterisation & Sensitivity List Fix

Module 09 – flopr.v | Parameterisation & Sensitivity List

✗ INITIAL (GitHub – Buggy)

 FINAL (Fixed – Working)

```
module flpr #(parameter WIDTH=8) (
    input clk, reset,
    input [WIDTH-1:0] d,
    output reg [WIDTH-1:0] q);
// Parameterised → reusable at any width
```

```
always @(posedge clk or posedge reset)
// 'or' keyword → IEEE Verilog-2001 standard
// Accepted by all tools without warning
```

```
if(reset) q<=0;
else q<=d;
// Same logic - now works for any WIDTH
// Used as flopr#(32) for PC register
```

```
`timescale 1ns / 1ps
```

Module 10

top.v

Top Module – Simulation vs FPGA Implementation Versions

Module 10 – top.v | Simulation Version (Commented in Final)

INITIAL (GitHub – Buggy)

```
module top(input clk, reset);
wire [31:0] PC,Instr,ReadData;
wire [31:0] WriteData,DataAdr;
wire MemWrite;
cpu cpu(clk,reset,PC,Instr,
    MemWrite,DataAdr,WriteData,ReadData);
imem imem(PC,Instr);
dmem dmem(clk,MemWrite,DataAdr,
    WriteData,ReadData);
// DataAdr used for memory address
// Works in simulation only
```

FINAL (Fixed – Working)

```
// SIMULATION VERSION (commented out in final)
////module top(input clk, reset);
////    wire [31:0] PC,Instr,ReadData;
////    wire [31:0] WriteData,DataAdr;
////    wire MemWrite;

cpu cpu(.clk(cpu_clk),.reset(reset),.PC(PC),
    .Instr(Instr),.MemWrite(MemWrite),
    .ALUResult(ALUResult), // ← correct port!
    .WriteData(WriteData),.ReadData(ReadData));
```

Module 10 – top.v | FPGA Implementation Version

✗ INITIAL (GitHub – Buggy)

```
module top(input clk, reset);
// Simple 2-port interface
// Cannot map to Arty A7 I/O

// No debounce logic
// Button bounce causes multi-step
// CPU advances unpredictably

// No clock management
// CPU runs at full speed
// Cannot observe individual steps

// No debug output
// Cannot observe CPU state on hardware
```

✓ FINAL (Fixed – Working)

```
module top(
    input CLK100MHZ,
    input [3:0] btn, sw,
    output [3:0] led,
    output led0_r,led0_g,led0_b,
    output led1_r,led1_g,led1_b);
// 20ms debounce at 100MHz (2,000,000 cycles)
reg [20:0] db_count;
reg db_stable, db_prev;
wire step_pulse = db_stable & ~db_prev;
// Rising edge = clean single step pulse

wire cpu_clk = step_pulse;
// CPU clock = one pulse per button press
// btn[1] = step, btn[0] = reset
// Human-observable execution!

// LED debug: sw[1:0] selects display
// 00→PC, 01→ALUResult, 10→Instr, 11→WriteData
assign led = selected_data;
assign led0_r=ALUResult[0]; // RGB LEDs too
```

Module 10 – top.v | Why the FPGA Version Needed a Complete Rewrite



Bug 1 – No FPGA board I/O

The initial top module had only 'clk' and 'reset' ports. An FPGA bitstream requires the top-level ports to match the constraint file's board pin assignments. With no matching ports, Vivado cannot route signals to physical I/O.



Fix – Arty A7-100T port mapping

The final top declares CLK100MHZ, btn[3:0], sw[3:0], led[3:0], and 6 RGB LED channels — exactly what the Arty A7 constraint file expects. Each port maps to a physical pin through the XDC constraint file.



Bug 2 – No button debounce → multiple steps

Mechanical buttons bounce 5-50ms when pressed. Without debounce, a single press generates 10-100+ transitions. The CPU would advance many steps per press, making step-by-step observation impossible.



Fix – 21-bit debounce counter (20ms)

A counter checks if the button level stays stable for 2,000,000 cycles (20ms at 100MHz). Only then does db_stable update. A rising-edge detector on db_stable generates exactly one step_pulse per press.



Bug 3 – ALUResult vs DataAdr port name mismatch

Initial top connected memory address as 'DataAdr', but the cpu module's output port is actually named 'ALUResult'. This name mismatch caused a port-not-found elaboration error in Vivado.



Fix – Named port connections + ALUResult

Final top uses named port connections (.ALUResult(ALUResult)) instead of positional, eliminating all name-mismatch risks. The data memory address correctly uses ALUResult as per the ARM microarchitecture.

Module 11

testbench.v

Testbench – Clock Initialization & Expected Value Fix

Module 11 – testbench.v | Initialization & Assertion Fixes

✗ INITIAL (GitHub – Buggy)

```
initial begin
    reset <= 1;
    #10 reset <= 0;
    clk <= 1; // ← clk init AFTER delay!
    // clk starts as X until this point
end

always #5 clk=~clk;
// If clk=X at t=0, ~X = X forever!
// Clock never oscillates → no simulation

$display("Test Failed: R5=%d, expected 7",
    dut.cpu.dp.rf.rf[5]);
// Wrong expected value printed! (7 vs 11)

// Uses non-blocking <= for initial block
// Non-blocking in initial is legal but
// can cause scheduling surprises at t=0
```

✓ FINAL (Fixed – Working)

```
initial begin
    clk = 0; // FIX: init clock at t=0
    reset = 1;
    #10 reset = 0;
end
// clk is known from t=0
always #5 clk = ~clk;
// With clk=0 at t=0:
// t=5→1, t=10→0, t=15→1 ...
// Clock oscillates correctly

$display("Test Failed: R5=%d, expected 11",
    dut.cpu.dp.rf.rf[5]);
// Correct expected value: 11

// Uses blocking = for initial block
// Blocking in initial block: values
// set immediately, no scheduling ambiguity
```

Module 12

condcheck.v

Condition Checker – Embedded in condlogic (New in Final)

Module 12 – condcheck.v | GE Flag & Full Condition Table

✗ INITIAL (GitHub – Buggy)

```
// condcheck NOT present as separate
// entity in initial code.
// It was referenced but never defined
// → 'module not found' elaboration error!
```

```
// No GE (greater-or-equal) signal
// Conditions GE, LT, GT, LE could not
// be evaluated correctly
```

```
// Missing many condition codes
// Only EQ,NE likely present
// ARM has 16 condition codes!
```

✓ FINAL (Fixed – Working)

```
module condcheck(
    input [3:0] Cond,
    input [3:0] Flags,
    output reg CondEx);
// Defined inline in condlogic.v

wire ge;
assign ge = ~(neg ^ overflow);
// ge=1 when N==V (ARM definition)
// Replaces old 'ge=(neg==overflow)'
case(Cond)
    4'b0000:CondEx=zero;    // EQ
    4'b0001:CondEx=~zero;  // NE
    4'b1010:CondEx=ge;     // GE
    4'b1100:CondEx=~zero&ge;// GT
    4'b1110:CondEx=1'b1;   // AL
    // ... all 16 ARM conditions
```

Performance & Functional Comparison: Initial vs Final

Metric	Initial (GitHub)	Final (Fixed)	STATUS
EDA Playground Simulation	FAILED – waveform not generated	PASSED – all waveforms correct	✓
Vivado Simulation	FAILED – multiple elaboration errors	PASSED – simulation runs clean	✓
FPGA Synthesis (Vivado)	FAILED – \$readmemh, port errors	PASSED – bitstream generated	✓
FPGA Implementation	NOT POSSIBLE – no board I/O	PASSED – Arty A7-100T deployed	✓
Clock Strategy	Free-running clk (uncontrolled)	Button-stepped (human observable)	✓
ALU Operations	All 4 ops gave WRONG results	ADD/SUB/AND/OR all correct	✓
Branch Instructions	Branch target WRONG (wrong SrcA)	Branch target correct (PC+8+offset)	✓
Conditional Execution	CondEx X-propagation → all X	All 16 ARM conditions functional	✓
Register File Reset	All regs X at startup	All regs cleared to 0 on reset	✓
Instruction Memory	X if memfile.dat not found	All 23 instructions pre-loaded	✓
Data Memory	Uninitialized (X propagation)	Initialised 0–63, no X	✓
Test Result (R5==11)	FAIL – clk never toggled (X)	PASS – R5 correctly = 32'd11	✓
Vivado DRC Warnings	Multiple critical warnings	Zero DRC errors or warnings	✓
FPGA Debug Output	None	LED shows PC/ALUResult/Instr/WData	✓

New Modules Added in Final (Not Present in Initial)

NEW: clk_divider.v

A simple clock divider providing a ~1Hz slow clock from the 100MHz board clock. Used during early development/testing before the button-step approach was finalized.

```
module clk_divider(input clk, output reg slow_clk);
    reg [25:0] count = 0;
    always @(posedge clk) begin
        count <= count + 1;
        slow_clk <= count[25]; // ~1Hz
    end
endmodule
```

Why needed:

The initial code had no clock management for FPGA. The clk_divider was an intermediate step to get visible behaviour before the button-debounce step approach was implemented.

NEW: constraint.xdc

Xilinx Design Constraints file mapping all top-level ports to physical Arty A7-100T board pins. Essential for any FPGA implementation — completely absent in the initial GitHub code.

```
set_property -dict {PACKAGE_PIN E3 IOSTANDARD LVCMOS33}
[get_ports {CLK100MHZ}];
set_property -dict {PACKAGE_PIN D9 IOSTANDARD LVCMOS33}
[get_ports {btn[0]}];
set_property -dict {PACKAGE_PIN H5 IOSTANDARD LVCMOS33}
[get_ports {led[0]}];
// ... all board I/O pins mapped
```

Why needed:

Without an XDC file, Vivado cannot assign ports to physical FPGA pins. The bitstream would have no connection to board buttons or LEDs, making the design unobservable on hardware.

NEW: memfile.mem

The ARM machine-code program stored as hex values. In the final design, these are loaded directly into imem.v's initial block. The .mem file serves as reference documentation of the test program.

```
E04F000F // RSB R0,PC,PC (get PC)
E2802005 // ADD R2,R0,#5
E280300C // ADD R3,R0,#12
...
E5802054 // STR R2,[R0,#84]
```

Why needed:

Provides human-readable documentation of the 23-instruction test program. Confirms that R5 should contain 11 after full execution, validating the testbench assertion.

Key Takeaways

Op-code Swap (ALU)

The most critical bug — all arithmetic was executing wrong operations. Fixing the 2-bit ALUControl mapping made the CPU functional.

Signal Type Correctness

Changing output reg → output (wire) on 20+ ports eliminated all multi-driver synthesis errors and latch inference warnings.

FPGA-ready Top Module

Button debounce, step-clock, constraint file and LED debug transforms a simulation-only design into a deployable FPGA system.

Branch Target Fix (Decoder)

Setting RegSrc=2'b01 for branch instructions correctly feeds PC+8 as SrcA, enabling correct PC-relative addressing.

Reset Propagation

Adding reset to regfile and initialising imem/dmem eliminated all X-propagation, making waveforms meaningful from cycle 0.

Result

Initial: 0/0 tests pass, waveform never generated. Final: All tests pass, deployed on Arty A7-100T with step-observable execution.